La caja negra

Diseño y arquitectura del reemplazo

Documento de diseño técnico para el reemplazo fiel de un servicio HTTP en producción sin código fuente ni documentación.

1. Contexto y objetivo

Un servicio HTTP lleva años en producción sin código fuente ni documentación. Lo usan clientes reales, mueve dinero real y sostiene operaciones reales. El objetivo es un reemplazo que, ante el mismo request, devuelva el mismo status y el mismo cuerpo que el original, errores incluidos, sin llamarlo.

Además de idéntico en el cable, el reemplazo tiene que ser operable: durable, seguro ante concurrencia, con credenciales revocables y con rastro auditable. Idéntico no alcanza si un reinicio pierde plata: los saldos son de los clientes, no nuestros. Cada token es un cliente con su propio estado, que ningún otro cliente puede leer ni modificar. Cuatro principios rigen el diseño:

Principios de diseño

1. **Fidelidad del cable sobre elegancia.** Las rarezas del original se conservan; no se corrige el protocolo.
2. **Pocas piezas.** Cada componente adicional es un punto de falla y se agrega solo lo que el sistema necesita.
3. **Lo no sondado se nombra como supuesto.** No se afirma un comportamiento sin una prueba repetida.
4. **El proceso no es la fuente de verdad.** El estado sobrevive al reinicio y no depende de qué instancia atiende el request.

2. Restricciones y supuestos

Restricciones del contrato que condicionan el diseño:

- Todas las llamadas llevan `Authorization: Bearer <token>`; con otro token o sin token, el reemplazo responde como el original.
- `POST /msg` recibe `{"m": "COMANDO;args;control"}`; `POST /__reset` vuelve al estado inicial; `GET /health` responde 200 cuando está listo; cualquier otra ruta responde exactamente como el original, incluidas las que no existen.
- Un test pasa con el mismo status y el mismo cuerpo: si ambos son JSON, el orden de las claves no importa, los arrays van en orden y los strings y números son idénticos; si alguno no es JSON, se compara el texto carácter por carácter. Los headers no se comparan y cada paso tiene 5 segundos.
- El reemplazo no puede llamar al original ni a otro reemplazo, y cada token personal admite 10 requests por segundo y 8000 por día contra el original.

Supuestos de diseño, con la razón y lo que pasa si resultan falsos:

Supuesto	Por qué se asume	Si resulta falso
Hay estado por cliente.	El reset existe y los ejemplos muestran saldos que cambian.	Sin estado el libro sobraría; hoy es la pieza que sostiene saldos y contadores.
El id del cliente sale del token.	Ningún ejemplo trae un id de cliente en el cuerpo.	Habría que leerlo del mensaje y el aislamiento por token no bastaría.
La cuenta solo existe dentro de un cliente.	El reset es por token y los ejemplos nombran `AC-` sin dueño.	Si `AC-0001` fuera global, la clave sería solo `cuenta_id` y habría colisiones.
El original distingue una falla interna de un error de negocio.	Producción de años y errores de negocio acotados a un catálogo chico: no hay un código de excepción entre ellos.	Nuestro 503 dejaría de tener sentido y se retiraría; hoy es supuesto.

Supuesto	Por qué se asume	Si resulta falso
Una sola región alcanza.	La latencia entre el API y la base entra en el presupuesto de 5 segundos por paso.	Habría que replicar por región y resolver el conflicto de escritura, que el diseño hoy evita al no admitir dos escritores.

3. Arquitectura

Cinco componentes y un proceso aparte. El dominio nunca construye HTTP: devuelve un resultado y el serializador lo convierte en la respuesta exacta del cable. El libro es el único que escribe SQL.

El diagrama de arquitectura está en la figura 1 y también como archivo aparte (`architecture.png`). El API no guarda estado propio: todo lo que hay que recordar vive en la base, así que la cantidad de procesos no cambia el comportamiento.

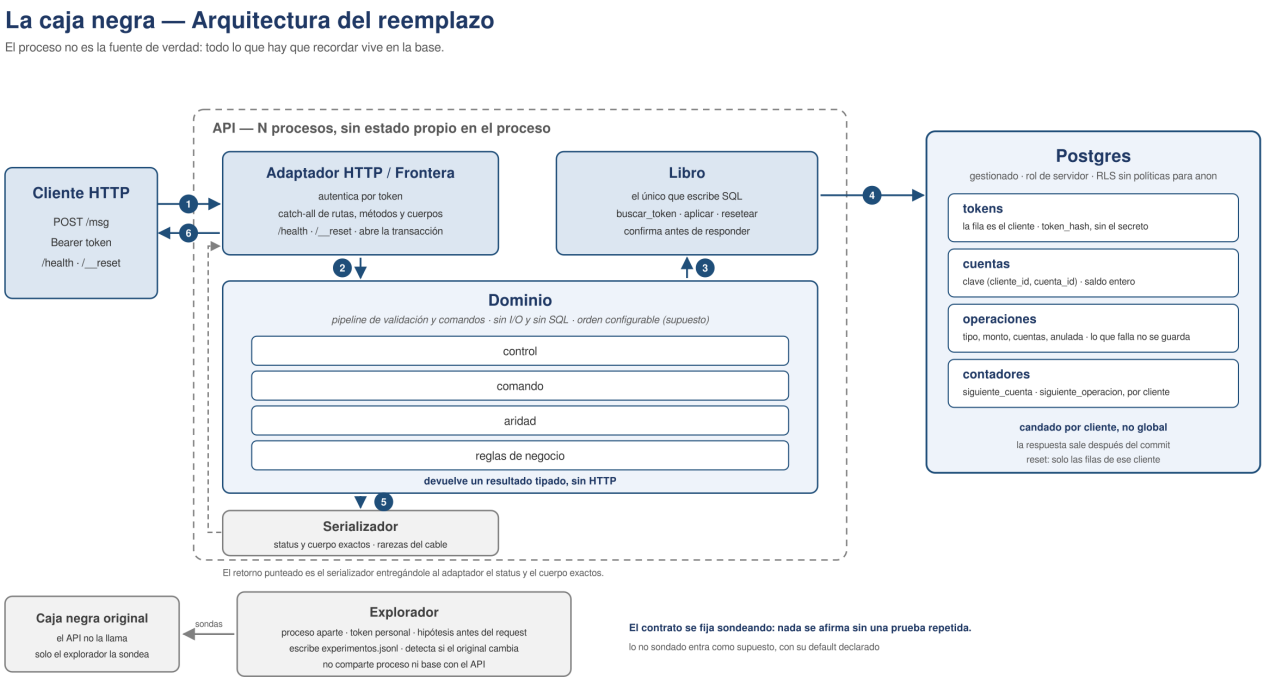


Figura 1 — Arquitectura de componentes

Flujo de un request:

1. El cliente envía `POST /msg` con su token; la frontera clasifica la ruta, autentica el token y abre una transacción.
2. El dominio ejecuta el pipeline y el comando contra el cliente autenticado, leyendo y escribiendo por el libro.
3. El libro confirma la transacción y el serializador convierte el resultado tipado en status y cuerpo exactos.
4. La frontera responde al cliente, ya confirmado el commit.

4. Diseño de los componentes

Cada componente tiene una responsabilidad y una regla que lo mantiene fiel al original.

4.1 Frontera / Adaptador HTTP

- Recibe toda ruta y método, autentica por token y no decide reglas de negocio; ningún default del framework llega al cliente.
- Resuelve lo no previsto con tabla propia: `método no permitido`, `tipo de contenido`, `JSON malformado`, `falta del campo m`, `barra final`, `HEAD`, `OPTIONS` y `GET` sobre `/msg`. Cada respuesta se copia del original y hasta sondearla es supuesto.

- Expone `/health` y `/__reset`, abre una transacción por request, delega en el dominio y responde después del commit.

4.2 Dominio

Recibe el mensaje y el cliente autenticado y devuelve un resultado tipado. No conoce SQL y no hace I/O, para poder probarlo sin base.

- Pipeline de validación como lista ordenada y configurable (control, comando, aridad y reglas de negocio). El orden real entre etapas es supuesto hasta sondearlo.
- Comandos `PING`, `ALTA`, `CONSULTA`, `DEPOSITO`, `TRANSFER` y `ANULA`.
- Límites como configuración, no como constantes dispersas. No se sabe si el tope es por operación o acumulado.
- Normalización: el comando no distingue mayúsculas.
- Operaciones reversibles: `ANULA` revierte y cada operación guarda lo necesario para deshacerse, más su bandera `anulada`.
- Contadores `AC-` y `OP-` que avanzan solo tras un éxito.

La función de control. Es el riesgo principal y estas son medidas, no intuiciones:

■ Función de control — Riesgo principal

- Es determinista y función pura del mensaje. Payloads repetidos dan el mismo control aunque el estado del cliente haya cambiado, así que no es un nonce ni un contador.
- No distingue mayúsculas: `CONSULTa;AC-0001;14` y `CONSULTA;AC-0001;14` dan el mismo control.
- No es un checksum barato. Se probaron offline unas 30 funciones sobre los 20 ejemplos —suma de ordinales, suma ponderada por posición, suma de dígitos, XOR, hashes polinómicos en base 7, 17, 31, 33 y 131, CRC32, Adler32, MD5, SHA1 y SHA256— cruzadas con cinco normalizaciones (crudo, mayúsculas, minúsculas, sin `;` y el mensaje completo), buscando módulo puro para cada `m` entre 2 y 1000 y transformaciones afines módulo 10, 100, 1000, 256, 97, 101 y 128. Cero coincidencias. Se aprende sondeando con el oráculo, no deduciendo.
- Mientras no esté, el default declarado es aceptar cualquier control numérico. Cuesta los pasos que prueban E202 a propósito, 1 de cada 20 en la muestra visible. Es un supuesto, va escrito como tal, y es mejor que rechazar todo.

El control es la compuerta. El umbral de la verificación es 20 de 20 sobre los ejemplos y cada uno de esos pasos necesita un control aceptado, así que el control es el paso 1 del orden de implementación, no el último.

4.3 Libro y serializador

El libro es el único que escribe SQL: el adaptador no escribe SQL y el dominio no conoce SQL. Expone `buscar_token(hash)`, que devuelve el token vigente con su fila o nada; `aplicar(cliente, comando, args)`, que ejecuta el comando dentro de la transacción; `resetear(cliente)`; y `base_responde()`, usado solo por el puerto interno de administración.

El serializador escribe respuestas literales, nunca construidas por concatenación, y conserva las rarezas del cable: `recieved`, `cuentaId` frente a `cuenta_id` y el espacio final del mensaje de comando desconocido. No se corrigen. La tabla de traducción vive acá, en código; el dominio no conoce status ni claves JSON.

Resultado	Status	Cuerpo
Pong	200	`{"pong": true}`
CuentaCreada(id)	201	`{"cuentaId": id}`
Saldo(id, monto)	200	`{"saldo": monto, "cuenta_id": id}`
OperacionAceptada(id)	200	`{"status": "recieved", "operacionId": id}`
Anulada(id)	200	`{"ok": true, "anulada": id}`
YaAnulada	200	`{"ok": false, "motivo": "ya anulada"}`
ControlMalFormado	400	`{"error": "E201", "mensaje": "control mal formado"}`

Resultado	Status	Cuerpo
ControlInvalido	400	`{"error": "E202", "mensaje": "control inválido"}`
ComandoDesconocido	400	`{"error": "E301", "mensaje": "comando desconocido "}`
AridadInvalida	400	`{"error": "E302", "mensaje": "aridad inválida"}`
CuentaInexistente	404	`{"error": "E401", "mensaje": "cuenta inexistente"}`
LimiteExcedido	422	`{"error": "E506", "mensaje": "límite excedido"}`
SaldoInsuficiente	422	`{"error": "E507", "mensaje": "saldo insuficiente"}`

`/health`, el reset, el token ajeno, el JSON roto y la ruta desconocida no son resultados del dominio.

4.4 Registro de tokens y modelo de datos

El registro es la fuente de verdad de las credenciales y se apoya en la misma base. El motivo sale del sistema, no del contrato: una empresa necesita revocar y rotar credenciales sin desplegar y saber quién creó cada una. No hay ruta pública de administración.

Cuatro tablas. El token no es una columna suelta en las cuentas: la fila de `tokens` es el cliente.

Tabla	Clave	Columnas	Notas
`tokens`	`id`	`token_hash` único, `etiqueta`, `created_at`, `created_by`, `expires_at`, `revoked_at`, `revoked_by`	No se guarda el secreto.
`cuentas`	`(cliente_id, cuenta_id)`	`saldo` entero, `nombre`	`cliente_id` apunta a `tokens.id`. La unicidad de `nombre` no está confirmada.
`operaciones`	`(cliente_id, operacion_id)`	tipo, monto, cuenta debitada, cuenta acreditada, `anulada`	Un depósito llena solo la acreditada; una transferencia, las dos. Lo que falla no se guarda.
`contadores`	`cliente_id`	`siguiente_cuenta`, `siguiente_operacion`, ambas en 1	Una fila por cliente. No es una secuencia global.

La clave de una cuenta es el cliente más el id: `AC-0001` puede existir en dos clientes y no es la misma fila. El token se guarda como HMAC-SHA256 con una pimienta que vive en el entorno, no es un token y no se rota. Vigente es no revocado y con `expires_at` en el futuro, en UTC; expirado o revocado se trata como ajeno. No hay caché.

4.5 Aislamiento, concurrencia y durabilidad

Un token es un cliente. El cliente no manda su id: sale del token vigente. Lo que es de otro cliente no se lee ni se modifica; si pide una cuenta ajena, para él no existe. La condición `cliente_id` va en cada consulta y cada escritura, y el reset filtra por el mismo valor.

- El dominio corre dentro de una transacción por request; la respuesta HTTP sale después del commit.
- Candado por cliente, no global. Se consideraron `pg_advisory_xact_lock` sobre el cliente y `SELECT … FOR UPDATE` sobre su fila de `contadores` y sus cuentas en orden determinista de `cuenta_id`. Se elige el advisory: toma un solo candado con la identidad del cliente, no exige que la fila de `contadores` exista todavía y elimina el deadlock por orden de filas.
- Dos clientes distintos operan en paralelo; dos movimientos del mismo cliente no se pisan. `PING` y la consulta no toman el candado de escritura: ven lo ya confirmado y no ven la mitad de un depósito.
- El token se relee dentro de la transacción: si lo revocaron, no se confirma. La espera del candado corta a los 4 segundos, porque el cliente abandona a los 5.
- No se reintenta un `POST /msg`: un reintento puede depositar dos veces.

4.6 Reset, escala y operación

`POST /__reset`, en una transacción, borra las cuentas y las operaciones de ese `cliente_id` y devuelve sus contadores a 1. No trunca: borraría a los demás. Sin token vigente no se toca nada. Es una palanca de operación, no una licencia para no persistir.

El estado no está en el proceso, así que sumar instancias no parte el estado: cualquier proceso atiende cualquier comando del cliente autenticado porque el estado está en la base; ese es el pago concreto de la base de datos. El crecimiento va por pasos: primero más procesos, después un pool de conexiones, después una réplica de solo lectura para lo que tolera atraso; el saldo se queda en el primario. Un standby que se promueve para sobrevivir a la caída de una región. Nunca dos escritores. No hay caché de saldos ni de tokens.

El contrato fija exactamente una ruta de salud y su respuesta debe ser la del original; por eso `/health` responde 200 cuando el proceso está listo, con el cuerpo fiel al original, y no hace un viaje de ida y vuelta por request. Una empresa igual quiere que el balanceador no mande tráfico a un proceso sin base: esa disposición se expone en un puerto interno de administración, nunca en una ruta nueva del contrato, porque el contrato no admite rutas nuevas. Es un trade-off explícito, fidelidad del contrato contra salud con dependencias, y se elige la fidelidad.

Si no hubo commit y la base no responde o el candado no sale, la respuesta es 503 con `{"error":"no-disponible"}`. Es un supuesto, no un hecho: no vimos la falla interna del original y no se le inventa cuerpo por analogía. No se usa un error de negocio ni un 200. El rastro es una línea por request en la salida del proceso y sobrevive al reset: lleva hora UTC, id nuestro, ruta, estado del token, id de la fila del token, comando, resultado, si hubo commit, duración y clase de falla, y no lleva `Authorization`, secreto, pimienta, cadena de conexión ni el `m` crudo. En una operación real esa salida va a un colector con retención.

En el entorno van `DATABASE_URL`, la pimienta del registro y nada más: el token del equipo se registra una vez y no vuelve a la configuración. La URL pública es la del API, en una plataforma que mantenga el proceso despierto y que no levante dos escritores.

La administración es un CLI, no una ruta: `crear`, `registrar`, `revocar` y `listar`. `registrar` lee el secreto por stdin y sirve para el token que los clientes ya tienen; `crear` inventa uno nuevo, para pruebas nuestras; `listar` no muestra secretos. El CLI no revoca el último token vigente y la rotación se solapa: primero el nuevo, después se revoca el viejo. Si la tabla está vacía al arrancar, se puede sembrar uno desde un secreto de un solo uso; después el proceso no lo necesita. El explorador es un componente cerrado, proceso aparte; no comparte proceso ni base con el API y no lo importa. Su detalle vive en `docs/explorer.md` y acá se lo nombra como el origen de la evidencia y del log de experimentos.

5. Decisiones y alternativas descartadas

Cada decisión está contrastada con la alternativa que se descartó y por qué.

Decisión	Se descarta	Por qué
Estado en Postgres, no en el proceso.	El mapa en memoria.	Los saldos son plata de clientes reales, un reinicio no puede perderlos, y con el estado en el proceso una segunda instancia atendería con otro estado.
Candado por cliente.	El lock global y el lock de filas sin orden.	Dos movimientos del mismo cliente no pueden leer el mismo saldo, y un lock global bloquea a los demás clientes.
Responder después del commit.	Responder antes de confirmar.	Un 200 sobre una transacción que después falla es plata que el cliente cree que movió.
Registro de tokens en la base con secreto hasheado.	La lista de tokens en una variable de entorno.	Una empresa necesita revocar y rotar sin desplegar y sin guardar el secreto.
CLI de administración.	Una ruta pública de administración.	El contrato no admite rutas nuevas y administrar credenciales por HTTP amplía la superficie de ataque.
El dominio devuelve un resultado y el serializador escribe el JSON.	Que el dominio arme el HTTP.	Alguien corregiría la rareza del cable.

Decisión	Se descarta	Por qué
Catch-all HTTP propio.	Los defaults del framework.	Los errores de ruta, método y cuerpo deben ser los del original, carácter por carácter.
Pipeline con orden configurable.	El orden fijo en el código.	El orden real de los errores no está verificado y puede cambiar.
Contadores que avanzan solo tras un éxito.	Consumir el id aunque la operación falle.	Los ejemplos muestran que un error no consume id. Probable hasta repetirlo.
Operaciones que guardan lo necesario para revertirse.	Anular solo marcando.	El rechazo por saldo insuficiente de los ejemplos solo tiene sentido si la anulación devolvió el saldo. Probable hasta repetirlo.
Saldos leídos de la base, sin caché.	La caché de saldos.	Un saldo viejo es plata mentida.
Conectar directo o por sesión.	El pooler en transaction mode.	Rompe los candados de sesión y el estado de la conexión.
Conexión desde el rol de servidor con RLS activo y sin políticas para `anon`.	Exponer la Data API.	El acceso directo de un cliente anónimo a las tablas del libro es una fuga entre clientes.

6. Casos difíciles

Situaciones que ponen a prueba el diseño y cómo las resuelve.

Caso	Cómo lo resuelve el diseño
Dos clientes con `AC-0001`	La clave es `(cliente_id, cuenta_id)`. Para el cliente B, la cuenta de A no existe.
Reset de un cliente	Borra solo las filas de ese `cliente_id` y devuelve sus contadores a 1. El otro conserva su saldo.
Dos débitos del mismo cliente	Candado por cliente: el segundo débito ve el saldo ya descontado y no hay doble débito.
Ruta, método o cuerpo que el original no prevé	Tabla propia de la frontera con la respuesta del original, no la del framework.
Reinicio del proceso	El estado está en la base; el saldo y los contadores siguen ahí. Nada que reconstruir.
El token se revoca a mitad de un request	El token se relee dentro de la transacción; si ya no está vigente, no se confirma.
El original cambia durante el día	Control, orden de errores y límites están concentrados para ajustarlos rápido. El cambio no se inventa: se detecta rejugando los ejemplos.
Token ausente o inválido	Respuesta del original, que hay que sondear. Hasta entonces es supuesto.

7. Riesgos y mitigaciones

Riesgo	Mitigación
El proceso se reinicia y pierde el estado.	Cerrado por diseño: el estado vive en la base, no en el proceso. La prueba de persistencia lo verifica.
La plataforma autoescala y levanta una segunda instancia.	Ya no parte el estado: cualquier instancia atiende contra la misma base. Solo hay que evitar dos escritores.
El hosting duerme el proceso por inactividad.	Elegir un plan sin suspensión y comprobar el tiempo de arranque contra el límite de 5 segundos.
La fórmula del control no se descubre.	Default declarado, la sonda como paso 1 y el resto de las pruebas bloqueado hasta tenerla.
El orden de errores, el tipo de tope y el token del set siguen siendo supuestos.	Mantenerlos como configuración y marcarlos como tales hasta sondearlos.
El candado espera más de la cuenta en una ráfaga.	Corte a los 4 segundos y 503, antes de los 5 del cliente.
RLS mal configurado deja ver filas ajenas.	RLS activo en las cuatro tablas, sin políticas para `anon`, y prueba de aislamiento con dos clientes.
El protocolo permite un depósito duplicado por reintento.	Riesgo abierto del protocolo; se detalla en la sección 9. No se cierra sin cambiar el cable.
El rejuego periódico agota el presupuesto del original.	Tope de 8 por segundo y conteo diario; `X-Frontier-Infra` no es evidencia.

8. Estrategia de verificación

El harness es un script propio: el servicio no llama al original, el script sí. Cada prueba tiene un umbral explícito y el servicio se da por cerrado solo cuando todas lo cumplen.

Prueba	Entrada	Esperado	Métrica y umbral
Ejemplos	Reset y los 20 requests de `ejemplos.jsonl` en orden	Mismo status y cuerpo que lo grabado, salvo cambios del servicio	Pasos que coinciden. Umbral: 20 de 20
Control	Barrido de la función sobre los 20 ejemplos y las sondas	Un control aceptado por paso	Es la compuerta: sin esto, el resto no arranca
Diferencial	La misma secuencia contra el original (token propio) y contra el reemplazo	Mismo status y cuerpo en cada paso	Pasos distintos sobre pasos totales. Umbral: 0
Detección de cambios	Rejuego de los 20 ejemplos contra el original cada 30 minutos	Sin diferencias; una diferencia dispara revisión	Diferencias por pasada. Umbral: 0
Aislamiento y reset	A crea `AC-0001` y B la consulta; A y B con saldo; reset de A	Para B no existe; B conserva el saldo y A vuelve a cero	Lecturas cruzadas y filas ajenas borradas. Umbral: 0
Concurrencia	Dos débitos simultáneos del mismo saldo, repetidos 100 veces	El segundo ve el saldo ya descontado	Dobles débitos. Umbral: 0
Persistencia	Reiniciar el proceso después de un depósito y antes de cualquier reset	El saldo y los contadores siguen ahí	Diferencias contra el estado previo. Umbral: 0
Ruta, cuerpo y token ausentes	Ruta inexistente, método no previsto, JSON malformado, falta de `m`, barra final, token ausente o inválido	La respuesta del original, no la del framework	Casos distintos sobre casos probados. Umbral: 0
Latencia	Los 20 ejemplos y el diferencial, midiendo cada paso	Muy por debajo del límite de 5 segundos	p95 por paso menor a 1 s y máximo menor a 5 s, ahora con la base en el camino
Salud	`GET /health` con el proceso arriba	200	Código y tiempo. Umbral: 200 en menos de 1 s

Presupuesto de requests al original:

- El token personal admite 10 por segundo y 8000 por día; el harness diferencial se limita a 8 por segundo para dejar margen.
- El rejuego de los 20 ejemplos cada 30 minutos cuesta unas 1000 requests al día en un solo token.
- Las respuestas con el header `X-Frontier-Infra` son de la infraestructura y no cuentan como diferencia.

9. Fuera de alcance

Lo que el diseño excluye a propósito, con su razón:

- Llamar al original desde el servicio. Si el original se apaga, el nuestro tiene que seguir, y el contrato lo prohíbe.
- Corregir `recieved` o unificar `cuentaId` con `cuenta_id`. El cliente ya habla así; corregirlo lo rompe.
- Una ruta pública para administrar tokens. El contrato no admite rutas nuevas; la administración es un CLI.
- Mejorar el protocolo. La igualdad incluye el error y la rareza.

Riesgo del protocolo que no se puede cerrar: el protocolo no trae una clave de idempotencia del cliente, porque el `OP-` lo emite el servicio. Un reintento del cliente puede depositar dos veces y el reemplazo no puede inventar una clave sin cambiar el cable. Queda como riesgo del protocolo, no como una regla de "no reintentar".

10. Orden de implementación

Ordenado por riesgo, no por capas. Primero lo que puede tumbar todo el trabajo.

1. **La función de control**, aislada, con las sondas que la aprenden. Es la compuerta: sin control aceptado no se reproduce ni uno de los 20 ejemplos.
2. **Esqueleto desplegable con la base**: autenticación, `/health`, `/__reset` y los 20 ejemplos. Desplegarlo temprano para que aparezcan las métricas públicas.
3. **Catch-all del adaptador y serializador** con respuestas literales.
4. **Dominio**: pipeline, comandos, transacción y candado por cliente.
5. **Registro de tokens y CLI.**
6. **Harness diferencial y rejuego periódico** de los ejemplos.
7. **Ajustar lo que las sondas muevan**: fórmula del control, orden de errores, límites.

Estructura

```
reto-2/
  api/
    domain/          pipeline y comandos, sin I/O y sin SQL
    adapter.py       frontera: rutas, auth, catch-all, /health, /__reset
    ledger.py        el único que escribe SQL
    registry.py      tokens
    serializer.py    status y cuerpo exactos
    cli.py           crear, registrar, revocar, listar
  migrations/        las cuatro tablas y las políticas de RLS
  explorer/          el sondeo: cliente, bitácora, corrida
  tests/
    api/             rejuego de los 20 ejemplos y comprobaciones puntuales
    explorer/        arnés del sondeo, con transporte falso y sin red
  tools/             build_pdf.py: docs/design.md -> docs/diseño_caja_negra.pdf
  docs/
```

Supuestos

No son reglas.

- Cuerpo de una ruta desconocida y de `método no permitido`, `tipo de contenido`, `JSON malformado`, `falta del campo m`, `barra final`, `HEAD`, `OPTIONS` y `GET` sobre `/msg`.

- `/health` sin token; cuerpo de `__reset`, con token vigente y sin él; cuerpo de `/msg` sin token vigente; JSON roto, `m` ausente o `m` que no es texto.
- Orden entre aridad, control y comando; fórmula del control; si el tope es por operación o acumulado.
- Cuerpo de `/health`.
- 503 cuando fallamos nosotros; que `ANULA` devuelva el saldo al revertir; que un error no consuma el id. Los ejemplos lo muestran; probable hasta repetirlo.